# INDEX

NAME: Jeevan Prabu STD.: _____ SEC.: _____ ROLL NO.: _____ SUB.: POAI

| S. No. | Date | Title | Page No. | Teacher's Sign / Remarks |
|---|---|---|---|---|
| 1. | 22/7/26 | Solve the 8 Queens problem | | |
| 2. | 22/7/26 | Depth first Search | | |
| 3. | 29/7/26 | Implementation of MINIMAX algorithm | | |
| 4. | 29/7/26 | Implementation of A* algorithm | | |
| 5. | 5/8/26 | Find minimum & maximum of two numbers | | |
| 6. | 30/9/26 | Implementation of decision making and knowledge representation. | | |
| 7. | 5/8/26 | Implementation of unification and resolution algorithm. | | |
| 8. | 12/8/26 | Implementation of Backward chaining | | |
| 9. | 19/8/26 | Implementation of forward chaining. | | |
| 10. | 9/9/26 | Implementation of blocks world program. | | |
| 11. | 16/9/26 | Implementation of a Fuzzy inference System. | | |

Ex.no: 8I
22/7/26

# 8- QUEENS PROBLEM

AIM:

To implement the 8-queens problem using backtracking in Python.

PROGRAM:

```
N = 8
board = [[0 for i in range(N)] for j in range(N)]
def is_safe(row, col):
    # left side
    for i in range(col):
        if board[row][i] == 1:
            return False

    # upper-left
    i = row
    j = col
    while i >= 0 and j >= 0:
        if board[i][j] == 1:
            return False
        i -= 1
        j -= 1
    # lower-left
    i = row
    j = col
    while i < N and j >= 0:
        if board[i][j] == 1:
            return False
```

```
        i += 1
        j -= 1
    return True

def solve(col):
    if col >= N:
        return True
    for row in range(N):
        if isSafe(row, col):
            board[row][col] = 1
            if solve(col+1):
                return True
            board[row][col] = 0
    return False

if solve(0):
    for row in board:
        print(row)
else:
    print("No Solution")
```

Output:

```
Q . . . . . . .
. . . . Q . . .
. . . . . . . Q
. . . . . Q . .
. . Q . . . . .
. . . . . . Q .
. Q . . . . . .
. . . Q . . . .
```

Ex.no: 2
22/7/26

# BREADTH FIRST SEARCH

AIM:

To implement Breadth - First Search Algorithm in Python Programming.

PROGRAM:

```
from collections import deque
graph = [[1, 2], [0, 2], [0, 1, 3, 4], [2], [2]]
visited = [False] * 5
queue = deque([0])
visited[0] = True
while queue:
    node = queue.popleft()
    print(node, end = " ")
    for i in graph[node]:
        if not visited[i]:
            visited[i] = True
            queue.append(i)
```

Output:

```
0 1 2 3 4
```

RESULT:

Thus the program is executed successfully.

Ex.no:3
29/7/26

# DEPTH FIRST SEARCH

AIM:

To implement Depth-First Search Algorithm in Python Programming.

PROGRAM:

```
def dfs(graph, node, visited):
    visited.add(node)
    print(node, end=" ")

    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)

graph = {
  'A' : ['B', 'C'],
  'B' : ['D', 'F'],
  'C' : ['F']
  'D' : [],
  'E' : ['F'],
  'F' : []
```

```
visited = set()
dfs(graph, 'A', visited)
```

OUTPUT:

A B D E F C

RESULT:

Thus the program is executed successfully.

Ex.No: 4

29/7/26 DEPTH LEVEL SEARCH

AIM:

To implement DLS to find a goal node in a graph tree.

PSEUDO CODE:

```
DLS (node, goal, unit)
    if node == goal;
        return success
    if limit == 0;
        return cutoff;
    for each neighbour of node:
        result = DLS (neighbour, goal, limit-1)
    if result => success
        return success
    return failure
```

RESULT:

Thus the DLS to find a goal node in a graph up to a specified depth limit

Ex.no : 5

5/8/26

# BI- DIRECTIONAL

AIM:

To find a path from start node to goal node by searching simultaneously from both directions

PSEUDO CODE:

Bidirectional Search (graph, start, goal):

Create two queues : $Q_1$, $Q_2$

Create two visited sets : $V_1$, $V_2$

add start to $Q_1$

add goal to $Q_2$

add start to $V_1$

add goal to $V_2$

while $Q_1$ and $Q_2$ are not empty:

node 1 = remove front of $Q_1$

add its unvisited neighbours to $Q_1$ & $V_1$

node 2 = remove front of $Q_2$

add its unvisited neighbours to $Q_2$ & $V_2$

if anynode is present in both $V_1$ & $V_2$

return success

return failure

RESULT:

Thus, a path is found using bi.-directional search.

30/7/26
Ex.no:6

# MINMAX SEARCH PROBLEM

AIM:

To implement a Minmax search program in Python for an AI game

PROGRAM:

```
def minimax (a, max-player)
    if len(a) == 1
        return a[0]
    mid = len(a) // 2
    left = minimax [a[:mid],
        not max-player)
    right = minimax (a[mid:],
        not max-player)
    if max-player:
        return max (left, right)
    else:
        return min (left, right)

a = [3, 5, 7, 12, 14, 1, 16, 8]
print (minimax (a, True))
```

Output: 14

RESULT: Thus a minimax search program was implemented using recursion and implemented successfully.

Ex. no. 7
5/6/26

# ALPHA - BETA PRUNING

AIM:
To implemented the Alpha - Beta Pruning algorithm using Python to reduces unnecessary searches.

PROGRAM:

```
def minimax (a, alpha, beta, max-player)
  if len(a) == 1:
      return a[0]
  mid = len(a) // 2
  left = min ; max (0, (: min), alpha, beta, not max_players)
  if max-players :
      alpha = max (alpha, left)

  else :
    beta    min ( beta, left)
  if alpha >= beta
      return left
  right = minmax (a[mid:], alpha, meta, not max-
  _player )
    return max ((left, right)
    if (max-player else min ( left , right)

  a = [ 3, 5, 2, 9]
print (minmax (a, float (inf'), float (inf'), True))
```

Output = 3

RESULT:
Thus Alpha-Beta pruning was implemented successfully.

```
print ("Unification:", unify ("x", "John"))
print ("Resolution:", resolve (["~P", "Q"], ["P"]))
```

Output:

Unification : ['x', 'John']

Resolution : ['Q']

Result:

Thus, the Unification and Resolution algorithms were implemented successfully using python.

Ex.no.9
19/6/26 IMPLEMENTATION OF BACKWARD CHAINING

Aim:

To implement the backward chaining algorithm using Python.

Program.

```
facts = ['a', 'b']
rules = [
    'd' = ['a', 'b'],
    'e' = ['b', 'c'],
    'f' = ['d', 'e'].
]

def backward_chaining(goal):
    if goal in facts:
        return True
    if goal in rules:
        return all(backward_chaining(x) for x
                   in rules[goal])
    return False

goal = 'f'
print("Goal achieved" if backward_chaining(goal) else
      "Goal not achieved")
```

```
visited = set()
dfs(graph, 'A', visited)
```

OUTPUT:

A B D E F C

RESULT:

Thus the program is executed successfully.

Ex.no: 4

29/7/26

# DEPTH LEVEL SEARCH

AIM:

To implement DLS to find a goal node in a graph tree.

PSEUDO CODE:

```
DLS (node, goal, limit)
    if node == goal;
        return success
    if limit == 0;
        return cutoff;
    for each neighbour of node:
        result = DLS (neighbour, goal, limit-1)
    if result => success
        return success
    return failure
```

RESULT:

Thus, the DLS to find a goal node in a graph up to a specified depth limit

Ex.no : 5
5/8/26

# BI- DIRECTIONAL

AIM:

To find a path from start node to goal node by searching simultaneously from both directions

PSEUDO CODE:

```
Bidirectional Search (graph, start, goal):
    Create two queues: Q1, Q2
    Create two visited sets: V1, V2
    add start to Q1
    add goal to Q2
    add start to V1
    add goal to V2
    while Q1 and Q2 are not empty:
        node1 = remove front of Q1
        add its unvisited neighbours to Q1 & V1
        node2 = remove front of Q2
        add its unvisited neighbours to
                Q2 & V2
        if anynode is present in both
            V1 & V2
            return success
    return failure
```

RESULT:

Thus, a path is found using bi.-directional search.

30/9/25 MINMAX SEARCH PROBLEM
Ex.no.6

AIM:
To implement a Minmax search program is Python for an AI game

PROGRAM:

```
def minimax (a, max-player)
    if len(a) == 1
        return a[0]
    mid = len(a) // 2
    left = minimax [a[:mid],
        not max-player)
    right = minimax (a[mid:],
        not max-player)
    if max-player:
        return max (left, right)
    else:
        return min (left, right)

a = [3, 5, 7, 12, 14, 1, 16, 8]
print (minimax (a, True)
```

Output: 14

RESULT: Thus a minimax search program was implemented using recursion and implemented successfully.

Ex. no. 7
5/6/26

# ALPHA - BETA PRUNING

AIM:
To implemented the Alpha - Beta Pruning algorithm using Python to reduces unnecessary searches.

PROGRAM:

```
def minimax (a, alpha, beta, max-player)
  if len(a) == 1:
      return a[0]
  mid = len(a) // 2
  left = min ; max (0, (:min), alpha, beta, not max-players)
  if max-players :
      alpha = max (alpha, left)

  else :
    beta  max ( beta, left)
  if alpha >= beta
      return left
  right = minmax (a[mid:], alpha, meta, not max-
  _players )
    return max ((left, right)
    if (max-player else min ( left , right)

  a = [ 3, 5, 2, 9]
print (minmax (a, float ('inf'), float ('inf'), True))
```

Output = 3

RESULT:
Thus Alpha-Beta pruning was implemented successfully.

Ex.no:8
12/8/24 IMPLEMENTATION OF UNIFICATION AND RESOLUTION ALGORITHM.

AIM:

To implement unification and resolution algorithm using Python

PROGRAM:

```
def unify(x, y, sub=[]):
    if x == y:
        return sub
    if isinstance(x, str) and x.islower():
        sub[x] = y
        return sub
    if isinstance(y, str) and y.islower():
        sub[y] = x
        return sub
    return none

def resolve(c1, c2):
    for x in c1:
        for y in c2:
            if x == '~' + y or y == '~' + x:
                return list(set(c1+c2) - {x,y})
    return none
```

Ex.no:8
12/6/24 IMPLEMENTATION OF UNIFICATION AND RESOLUTION ALGORITHM

AIM:

To implement unification and resolution algorithm using Python.

PROGRAM:

```
def unify(x, y, sub=[]):
    if x == y:
        return sub
    if isinstance(x, str) and x.islower():
        sub[x] = y
        return sub
    if isinstance(y, str) and y.islower():
        sub[y] = x
        return sub
    return none

def resolve(c1, c2):
    for x in c1:
        for y in c2:
            if x == '~' + y or y == '~' + x:
                return list(set(c1+c2) - {x,y})
    return none
```

```
print ("Unification :", unify ("x", "John"))
print ("Resolution :", resolve (["~P", "Q"], ["P"]))
```

Output:

Unification : ['x', 'John']

Resolution : ['Q']

Result:

Thus, the Unification and Resolution algorithms were implemented successfully using Python.

Ex. no. 9
19/8/25 IMPLEMENTATION OF BACKWARD CHAINING

Aim:

To implement the backward channing algorithm using python.

Program.

```
facts = ['a', 'b']
rules = [
    'd' = ['a', 'b'],
    'e' = ['b', 'c'],
    'f' = ['d', 'e'].
]

def backward_chaining(goal):
    if goal in facts:
        return True
    if goal in rules:
        return all(backward_chaining(x) for x
                   in rules[goal])
    return False

goal = 'd'
print("Goal achieved" if backward_chaining(goal) else
      "Goal not achieved")
```

Output:

Goal achieved

Result:

Thus, the Backward Chaining algorithm was implemented successfully using Python.

Ex.no: 10
9/9/26

# IMPLIMENTION OF FORWARD CHAINING

AIM:

To implement the forward chainning algorithm using python.

PROGRAM:

```
facts = {'a', 'b'}

rules = {
    'd': ['a', 'b']
    'e': ['b', 'c']
    'f': ['d', 'e']
}

def forward_chaining():
    changed = True
    while changed:
        changed = False
        for goal, conditions in rules.items():
            if all(x in facts for x conditions) and goal
                                    not in facts:
                facts.add(goal)
                changed = True
    print("Derived facts:", facts)

forward_chaining()
```

Output:

Derived facts: {'a', 'b', 'd'}

Result:

Thus, the Forward Chaining algorithm was implemented successfully using Python.

Ex.no:11
16/9/2024

# Implementation of a Fuzzy inference System

Aim:

To implement a fuzzy inference system for temp-based fan speed control.

Code:

```
import numpy as np.
import skfuzzy as fuzzy
tem = np.arange(0,41,1)
fanspeed = np.arange(0,11,1)
temp_low = fuzz.trapmf(temp,[0,10,20])
temp_medium = fuzz.trapmf[(fan_speed(0,0,5)]
fan_high = fuzz_point(fan_speed[5,10,10])
    t=28
= fuzz.interp_membership(temp, temp_low, t)
```

O/p:

```
fuzz.defuzz(fan_speed, aggregate, (centroid)
print('Temp:','t','(')
print("Fanspeed": round(o/p:2))
```

Result:

The fuzzy Inference System was Implemented to find the i/p temperature

Result:

The fuzzy Inference system was Implemented to find fan speed for the i/p temperature